

UCCD 1024
DATA STRUCTURES AND
ALGORITHMIC PROBLEM SOLVING

MID – TERM TEST

MAY 2021

DURATION: 1 HOUR 15 MINUTES

Answers ALL the questions in this paper. Read the instructions given to you before you start the test. Each question carries 20 marks.

1. We can use a 2-Dimensional Linked List to represent a mother with their children. The diagram in Appendix A1 shows the representation of this linked list. The struct definitions needed for this representation can be found in Appendices A2, A3 and A4. The main() function and the sample text file used (children.txt) together with the sample output is given in Appendix A5. Mother list is sorted according to name and children list is sorted according to age.
 - (a) Write a function **bool insertChild(const char *filename, List_Mother *list)** to insert the children store in text file to the mother. A sample of the text file, “children.txt” is given in Appendix A5. A child is matched to the mother according to the mother’s name. Function return false for empty list or file cannot opened.
 - (b) Write a function **bool printList(List_Mother list)** to print all the mothers and their children in the list to the screen. A sample of the output is given in Appendix A5. Function return false for empty list.
 - (c) Briefly explain the different between general linked list and stack. Provide an example on how to use each one in real life to solve problem.

2. (a) Answer the following question based on the segment of code below.

<pre> int main() { Stack s1, s2; char *name[8] = {"Orange", "Banana", "Apple", "Durian", "Avocado", "Olive", "Kiwi", "Papaya"}; char c[5] = {'A', 'E', 'T', 'O', 'U'}; char *str1, *str2; bool found; for(int x=0; x<8; x++){ found = false; for(int y=0; y<5; y++){ if(name[x][0] == c[y]){ s1.push(name[x]); found = true; } } if(!found) s2.push(name[x]); } //SECTION A – Draw the content //of s1 and s2 after the nested for loop. </pre>	<pre> //continue from first column cout << "\n\nStack 1: \n"; while(!s1.empty()){ if(s1.pop(str1)) cout << str1 << "\t"; } cout << "\n\nStack 2: \n"; while(!s2.empty()){ if(s2.pop(str2)) cout << str2 << "\t"; } </pre>
---	---

- (i) Redraw the table below to show the content (the item value in each node) of stacks s1 and s2 after the code is executed up to the comment with “SECTION A”. You can add more column or delete column if it is needed.

	1 st Node	2 nd Node	3 rd Node	4 th Node	5 th Node
s1					
s2					

- (ii) What is the output of this program?

- (b) Briefly explain why stack has the property **Last In First Out (LIFO)**.

3. (a) Answer the following question based on the segment of code below.

```
Queue queue;
string temp;
string name[6] = {"Mary", "John", "Alice", "Ken", "Mark",
                 "Jane"};

for(int i=0; i<6; i++)
    queue.enqueue(name[i]);

//SECTION A – Draw content of queue after the for loop

while(queue.size() > 1){
    for(int i=1; i<=3; i++){
        queue.dequeue(temp);
        if(i < 3)
            queue.enqueue(temp);
        else
            cout << temp << " is out of the queue.\n";
    }
}
queue.dequeue(temp);
```

- (i) Redraw the table below to show the content (the item value in each node) of the queue after the code is executed up to the comment with “SECTION A”. You can add more column or delete column if it is needed.

	1 st Node	2 nd Node	3 rd Node	4 th Node	5 th Node
queue					

- (ii) What is the output of this program?

- (b) Briefly explain why queue has the property **First In First Out (FIFO)**.

4. (a) The function to find a greatest common divisor (gcd) is given below. For example, gcd for 10 and 25 will be 5.

```
int gcd(int a, int b){  
    if(b==0)  
        return a;  
    if(a==0)  
        return b;  
    return (gcd(b, a%b));  
}
```

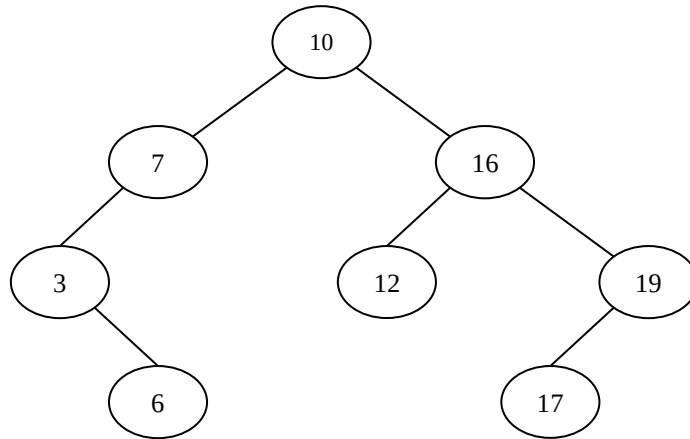
- (i) What are the base case and recursive case in the gcd function?
- (ii) What is the value of **result** and how many times the gcd function will be called (including first function call) if the function is called as below:

result = gcd(3, 26);

- (b) Describe what are the base case and the recursive case for the following problems if each problem is solved using recursion.
- (i) To sum up the integer value in every node in a linked list.
- (ii) To find the maximum integer in an array.

5. Answer the following questions related to trees.

- (a) Binary Search Tree 1 are given below. Answer the following questions based on this BST.



Binary Search Tree 1

- (i) Which of the following insertion order of nodes will create Binary Search Tree 1 above? You can choose more than one.

INSERTION ORDER 1 - 10 7 16 6 12 19 3 17

INSERTION ORDER 2 - 10 7 3 6 16 19 17 12

INSERTION ORDER 3 - 10 3 6 7 16 19 12 17

INSERTION ORDER 4 - 10 16 12 7 19 3 17 6

INSERTION ORDER 5 - 10 7 16 3 12 19 6 17

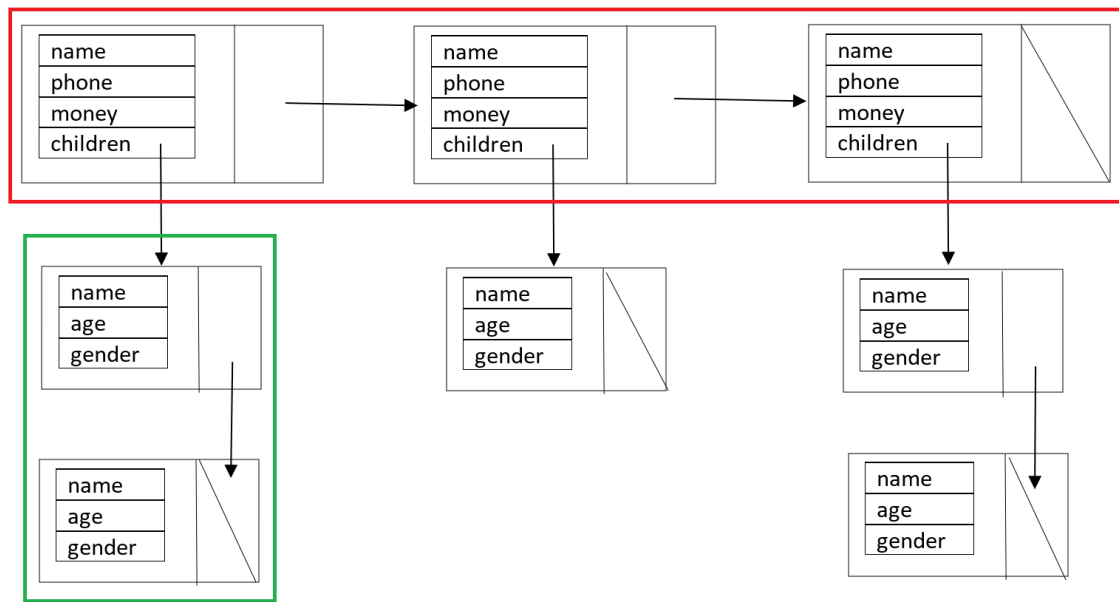
INSERTION ORDER 6 - 10 16 12 19 17 7 6 3

- (ii) List the order of the nodes visited using Inorder traversal.
- (iii) Draw the resulting BST if the following values are inserted in the order given to Binary Search Tree 1 above.
1 8 3 15 20 23
- (iv) List out one of the longest external paths in the BST you drawn in 5(a)(iii).

- (b) What will be printed if the Binary Search Tree 1 in Q5(a) above (before insert extra values) is used to call the function **guessWhat** function below? Assume that cur is pointing to root and value is 15 when the function is called.

```
void BST::guessWhat(BTNode *cur, int value) {  
  
    if (cur == NULL)  
        return;  
  
    if (cur->item > value) {  
        guessWhat (cur->left, value);  
        cout << cur->item << " ";  
    }  
  
    guessWhat(cur->right, value);  
}
```

Appendix A1: A 2 Dimensional Linked List Storing Mother and her Children



Explanation:

	Defined in List_Mother.h to store each mother in a node. The item for each node is defined in Mother.h. This list is sorted according to name.
	Defined in List_Child.h to represent the children list store in each mother node. The item for each node is defined in Child.h. This list is sorted according to age.

Appendix A2: Definition for Mother.h and Child.h

<pre>//Mother.h #ifndef Mother_type #define Mother_type #include <iostream> #include "List_Child.h" using namespace std; struct Mother { char name[20], phone[20]; double money; //List of children for mother List_Child children; bool compareName1(Mother); //using >= bool compareName2(Mother); //using == void print(); /* print() will print this format: Name: Audrey Phone: 0118973566 Money: 6000 */ }; #endif</pre>	<pre>//Child.h #ifndef Child_type #define Child_type struct Child { char name[30]; int age; char gender; //'M' or 'F' bool compareAge(Child); //using >= bool compareName(Child); //using == void print(); /* print() will print this format: Name: Louis Age: 6 Gender: M */ }; #endif</pre>
--	--

Appendix A3: Definition for Node_Mother.h and List_Mother.h

<pre>//Node_Mother.h #ifndef Node_Mother_type #define Node_Mother_type #include "Mother.h" using type = Mother; struct Node_Mother{ type item; Node_Mother *next; Node_Mother(type); }; #endif</pre>	<pre>//List_Mother.h #ifndef List_type #define List_type #include "Node_Mother.h" struct List_Mother { int count; Node_Mother *head; Node_Mother *find(int); List_Mother(); bool empty(); int size(); bool get(int, type&); bool set(int, type); bool insert(int, type); bool remove(int); //sort according to mother's name bool insert(type); }; #endif</pre>
---	---

Appendix A4: Definition for Node_Child.h and List_Child.h

<pre>//Node_Child.h #ifndef Node_Child_type #define Node_Child_type #include "Child.h" using type2 = Child; struct Node_Child { type2 item; Node_Child *next; Node_Child(type2); }; #endif</pre>	<pre>//List_Child.h #ifndef List_Child_type #define List_Child_type #include "Node_Child.h" struct List_Child { int count; Node_Child *head; Node_Child *find(int); List_Child(); bool empty(); int size(); bool get(int, type2&); bool set(int, type2); bool insert(int, type2); bool remove(int); //sort according to child's age bool insert(type2); }; #endif</pre>
---	---

Appendix A5: main() Function with Sample Text File (children.txt) and Sample Output

```
int main() {  
  
    List_Mother mother;  
    Mother m[5] = { {"Mary", "0123562789", 3000.00},  
                    {"Shelley", "0197621452", 2000.00},  
                    {"Audrey", "0118973566", 6000.00} };  
  
    //insert nodes to mother list  
    for (int i = 0; i < 3; i++)  
        mother.insert(m[i]);  
  
    //insert child to mother  
    if (insertChild("children.txt", &mother))  
    {  
        cout << "\n\nBelow are the children for each mother:\n";  
        if (!printList(mother))  
            cout << "\n\nCannot print from an empty list.\n\n";  
    }  
    else  
        cout << "\n\nCannot insert child to mother.\n\n";  
}
```

<pre>//children.txt //mother_name child_name age gender Shelley Beth 8 F Mary Sam 11 M Audrey Louis 6 M Audrey Kathy 9 F Shelley Jessy 10 F</pre>	<pre>//Sample output Below are the children for each mother: Mother: Name: Audrey Phone: 0118973566 Money: 6000 Below are children of Audrey Child 1 Name: Louis Age: 6 Gender: M Child 2 Name: Kathy Age: 9 Gender: F</pre>
--	---

Mother:
Name: Mary
Phone: 0123562789
Money: 3000

Below are children of Mary

Child 1
Name: Sam
Age: 11
Gender: M

Mother:
Name: Shelley
Phone: 0197621452
Money: 2000

Below are children of Shelley

Child 1
Name: Beth
Age: 8
Gender: F

Child 2
Name: Jessy
Age: 10
Gender: F

Press any key to continue . . .